Exercise Sheet 3 (programming part)

Machine Learning for Data Science, Winter Semester 2026/27 — Prof. Dr.-Ing. Gerhard Wunder, Freie Universität Berlin

Students: Furkan Dursun, Maxim Zotov, Lewin Ahlgrimm

We first import the relevant libraries.

In [1]:
```
import numpy
import sklearn
import sklearn.datasets
import matplotlib
%matplotlib inline
from matplotlib import pyplot as plt
```

In this exercise we perform a principal component analysis of the Iris dataset, a famous dataset from Fisher (1936) which one can access from sklearn. Each instance of the dataset is an iris plant which comes with four measurements (1. sepal length in cm, 2. sepal width in cm, 3. petal length in cm, 4. petal width in cm).

In addition to these measurements, the dataset also includes for each instance the type of iris plant (iris setosa, iris versicolour, iris virginica) which we treat here as metadata. Overall, the Iris dataset has 150 instances, and can be stored as an array of size 150 x 4.

The following cell loads the dataset. Additionally, some logarithmic transformation of the input features is performed so that a difference between e.g. 5mm and 6mm is given roughly the same importance as a difference between 5cm and 6cm. Such log-scaling gives more focus on features measuring smaller objects, compared to an approach based on the raw measurements. Also, we add a small increment of 1mm before applying the log-transform to ignore very small quantities that cannot be precisely measured.

In [2]:
```
dataset = sklearn.datasets.load_iris()

X = numpy.log(0.1+dataset['data'])
T = dataset['target']

target_names = dataset['target_names']
feature_names = dataset['feature_names']

X = X - X.mean(axis=0)

# N is # of rows, d is # of columns
N,d = X.shape
```

Note that one must keep in mind that our features have been log-transformed in any of the subsequent analyses and interpretations. A first basic analysis that can be performed is to measure how much dispersion there is in the data. The total variance of the data can be computed by the code below:

In [3]:
```
stot = (X**2).mean(axis=0).sum()
print('Total variance: %.3f'%stot)
```

```
Total variance: 1.027
```

Exercise 3: Principal Component Analysis (10+10+10 P)

We now would like to shed more light into the data by performing a principal component analysis (PCA). The exercise will consist of implementing PCA and then perform various analyses based on the learned model.

Implementing PCA

The simplest way of implementing PCA, is through an eigendecomposition of the data covariance matrix Σ, followed by solving the eigenvalue equation $\Sigma \boldsymbol{u} = \lambda \boldsymbol{u}$.

(a) Implement the PCA algorithm. Your code should return a tuple consisting of (1) a vector `L` containing the d eigenvalues, in decreasing order, and (2) a matrix `U` where each column contains the eigenvector associated to the eigenvalue.

Eigenvectors in this matrix should be ordered according to the eigenvalues, i.e. in a way that, for all indices $i = 1 \dots d$, the column `U[:,i]` contains the eigenvector associated to the eigenvalue `L[i]`.

In [4]:
```
def PCA(X):

    # Compute the covariance matrix.
    covariance = (X.T @ X) / N

    # Find eigenvalues and eigenvectors of the covariance matrix. (eigenvalues is in increasi
    eigenvalues, eigenvectors = numpy.linalg.eigh(covariance)

    # Sort the eigenvalues in decreasing order and reorder the eigenvectors in the same way
    ordered_indices = numpy.argsort(eigenvalues)[::-1]
    L = eigenvalues[ordered_indices]
    U = eigenvectors[:, ordered_indices]

    return L, U

L,U = PCA(X)

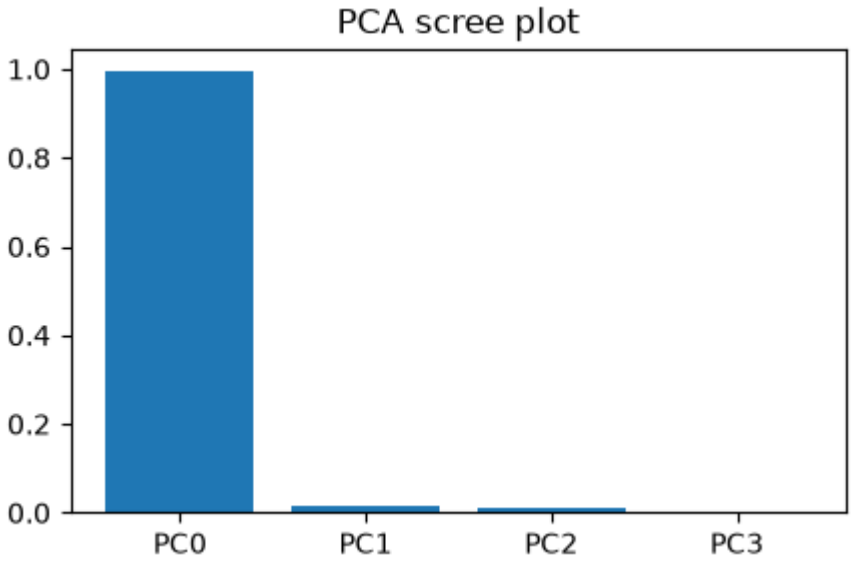
print(f'Eigenvalues:\n{L}')
print(f'Eigenvectors:\n{U}')
```

```
Eigenvalues:
[0.99354261 0.01743562 0.01291902 0.00266183]
Eigenvectors:
[[-0.11489579 -0.37596276 -0.35613464  0.84771403]
 [ 0.06357822 -0.91446113 -0.00630853 -0.39959837]
 [-0.56082242  0.1142948  -0.74665504 -0.33900025]
 [-0.81745641 -0.09669306  0.56181365  0.08234618]]
```

As mentioned in the lecture, PCA can be seen as a way of decomposing the total variance in the data. This decomposition of variance offered by PCA can be rendered in a PCA scree plot. (The following code further verifies that the PCA scree plot corresponds to a decomposition of the total variance, i.e. that the sum of plotted eigenvalues should be equivalent to the measured total variance.)

In [5]:
```
plt.figure(figsize=(5,3))
plt.title ('PCA scree plot')
plt.bar(numpy.arange(d),L)
plt.xticks(range(d),labels=['PC%d'%i for i in range(d)])
plt.show()

print('Total variance: %.3f'%stot)
print('Sum of eigenvalues: %.3f'%L.sum())
```




```
Total variance: 1.027
Sum of eigenvalues: 1.027
```

Expected output:

```
Total variance: 1.027
Sum of eigenvalues: 1.027
```

We observe that the first principal component captures most of the variance in the data. This suggests that one or a few strongly covariate features capture most of the variations in the data.

Understanding the First Principal Component

We now would like to gain understanding of this highly explanatory first principal component, specifically, to which input features this component responds. For this, we recall that the projection of data on a principal component has the dot-product form

$$z = \boldsymbol{u}^\top \boldsymbol{x} = u_1 x_1 + u_2 x_2 + \cdots + u_d x_d$$

where in our case, $\boldsymbol{u}$ is the principal component stored in our column vector `U[:,0]`.

(b) Write code that displays this projection operation instantiated to our particular problem formulation and PCA model.

In [6]:
```python
def printformula(U, feature_names):

    # Print z = sum_i u_i * log(0.1 + feature_i) with the coefficients of the first principal

    first_pc = U[:, 0]
    print("z = ")
    for i in range(len(feature_names)):
        print(f" {first_pc[i]:+.3f} * log(0.1 + {feature_names[i]})")

printformula(U, feature_names)
```

```
z =
 -0.115 * log(0.1 + sepal length (cm))
 +0.064 * log(0.1 + sepal width (cm))
 -0.561 * log(0.1 + petal length (cm))
 -0.817 * log(0.1 + petal width (cm))
```

Expected output (up to the sign of the eigenvector):

```
z =
 -0.115 * log(0.1 + sepal length (cm))
```

```
+0.064 * log(0.1 + sepal width (cm))
-0.561 * log(0.1 + petal length (cm))
-0.817 * log(0.1 + petal width (cm))
```

Each coefficient in the output is an entry of the first principal component `U[:,0]`, and each `log(0.1 + ...)` term is one of our log-transformed features. The code simply pairs every coefficient with its feature name and prints one term per feature.

As can be observed from the formula above, the first principal component responds mostly to a combination of petal length and petal width. It is on the other hand rather insensitive to sepal-related features.

Understanding the Data Projected on the First Principal Component

We now would like to get understanding of the data projected on this principal component. The main advantage of looking at the PCA space instead of the feature space is that its dimensionality has been reduced, thereby allowing for using tools such as simple histograms without performing a priori feature selection. The following code projects our data on the first principal component:

In [7]:
```
Z = X.dot(U[:,0])
```

(c) Write code that visualizes the projected data in the form of a histogram. Specifically, your visualization should split data according to the meta-data (here, the distinct plant species), and produce for each of them a distinct histogram with a specific color. *(Hint: use `plt.hist` with `alpha=0.5` and a common set of bins.)*

In [8]:
```
def PCAplot(Z, T, target_names):

    # Common set of bins. Compute the bin edges once and reuse them
    bins = numpy.linspace(Z.min(), Z.max(), 30)

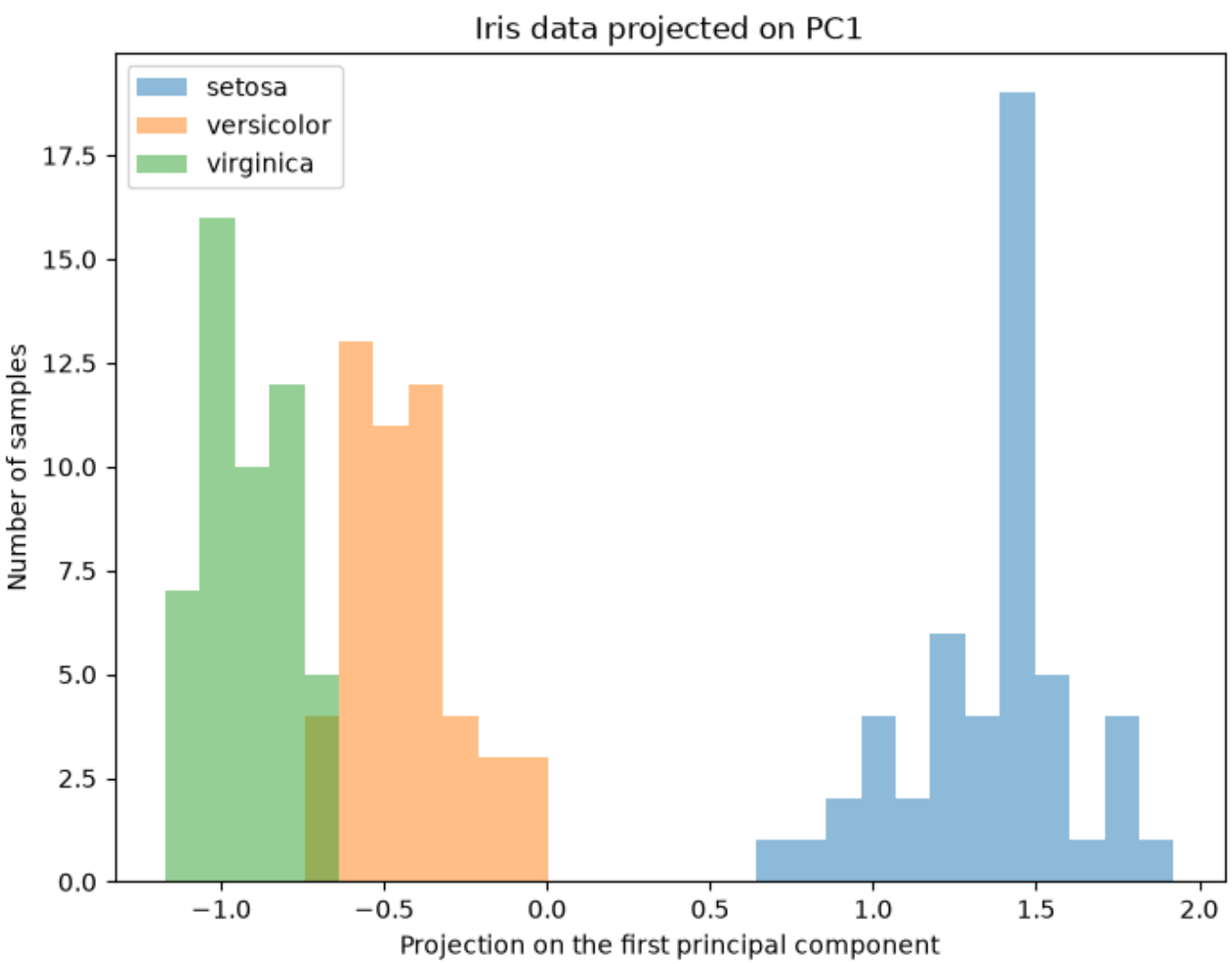
    plt.figure(figsize=(8, 6))

    # Draw one histogram for each species.
    for species in range(len(target_names)):
        # Keep only the projected values that belong to this species.
        species_data = Z[T == species]

        plt.hist(species_data, bins=bins, alpha=0.5, label=target_names[species])

    plt.xlabel('Projection on the first principal component')
    plt.ylabel('Number of samples')
    plt.title('Iris data projected on PC1')
    plt.legend()
    plt.show()

PCAplot(Z,T,target_names)
```



From this analysis, we can observe that the principal component represent meaningful variations in the data, in particular, species appear to be separable. The Iris setosa is separated from the other two species by a large margin, whereas the two other species are also separated to a certain extent, but not fully.